



Date: __ / __ / __

Experiment 7

7. a. You are a thief planning to rob a store. The store contains various items, each with its own weight and value. However, your knapsack can only carry a limited weight capacity. Given the weights and values of each item, determine the maximum total value you can obtain without exceeding the weight capacity of your knapsack.

You are given the following information:

- The weight capacity of your knapsack: W (an integer).
- The number of items available in the store: n (an integer).
- For each item i (where $1 \leq i \leq n$):
 - The weight of the i -th item: $wt[i]$ (an integer).
 - The value of the i -th item: $val[i]$ (an integer).

Write a C program that implements a solution to find the maximum total value that can be obtained from the items without exceeding the weight capacity of the knapsack, using Dynamic Programming.

Type of Lab Activity: Structured Enquiry

Aim	Material	Methods	Answer
Given	Given in Part	Given in Part	Open

Methodology

Construct the K Table (matrix) as given below using the below equation.

$$F(i, j) = \begin{cases} \max\{F(i-1, j), v_i + F(i-1, j-w_i)\} & \text{if } j-w_i \geq 0, \\ F(i-1, j) & \text{if } j-w_i < 0. \end{cases}$$

$$F(0, j) = 0 \text{ for } j \geq 0 \quad \text{and} \quad F(i, 0) = 0 \text{ for } i \geq 0.$$

		capacity j						
		i	0	1	2	3	4	5
$w_1 = 2, v_1 = 12$ $w_2 = 1, v_2 = 10$ $w_3 = 3, v_3 = 20$ $w_4 = 2, v_4 = 15$	0	0	0	0	0	0	0	0
	1	0	0	12	12	12	12	12
	2	0	10	12	22	22	22	22
	3	0	10	12	22	30	32	32
	4	0	10	15	25	30	37	37



Program

(Only for faculty reference, should not be shared to students)

```
#include<stdio.h>

// Function to find maximum of two integers
int max(int a, int b)
{
    return (a > b) ? a : b;
}

// Function to solve 0/1 Knapsack problem using Dynamic Programming
int knapsack(int W, int wt[], int val[], int n)
{
    int i, w;
    int K[n + 1][W + 1];

    // Build K[][] table
    for (i = 0; i <= n; i++)
    {
        for (w = 0; w <= W; w++)
        {
            if (i == 0 || w == 0)
                K[i][w] = 0;
            else if (wt[i - 1] <= w)
                K[i][w] = max(val[i - 1] + K[i - 1][w - wt[i - 1]], K[i - 1][w]);
            else
                K[i][w] = K[i - 1][w];
        }
    }

    return K[n][W];
}

// Main function
int main()
{
    int val[] = {12, 10, 20, 15};
    int wt[] = {2, 1, 3, 2};
    int W = 5;
    int n = sizeof(val) / sizeof(val[0]);
    printf("Maximum value that can be obtained is %d\n", knapsack(W, wt, val, n));
    return 0;
}
```



Sample Output

Maximum value that can be obtained is 37